

Jobsheet 5 – JavaScript DOM & Event

Sub-CPMK: Menerapkan manipulasi DOM & event JavaScript.

Perubahan dari Jobsheet 4

- Tambah `assets/js/app.js`.
- Hamburger menu: checkbox hack (CSS) diganti tombol + JS (`nav.classList.toggle("nav-open")`).
- Form Tambah Buku & Tambah Anggota: validasi client-side (`initValidasiForm`) – field wajib, rentang tahun, stok non-negatif – pesan error tampil inline via manipulasi DOM (`insertAdjacentElement`).
- Tabel Daftar Buku & Daftar Anggota: kolom pencarian real-time (`initTableFilter`) yang menyaring baris via `keyup`.
- Tombol Hapus (`.btn-hapus`): menampilkan `confirm()` lalu menghapus baris dari tampilan (masih front-end saja, belum ke server).

Cara menjalankan

Buka `index.html` di browser. Coba: submit form kosong (muncul error), ketik di kolom cari (tabel tersaring), klik Hapus (muncul konfirmasi).

Catatan

- Validasi di sini murni client-side dan bisa dilewati (nonaktifkan JS). Validasi server-side ditambahkan di Jobsheet 7 sebagai lapisan kedua yang wajib.
- Hapus baris di jobsheet ini hanya menghilangkan dari tampilan (belum persisten) – akan diganti proses hapus sungguhan ke database mulai Jobsheet 9.

Wireframe & User Flow – SIMPUS-Mini

Sub-CPMK: Merancang UI/UX aplikasi (proyek).

Halaman yang sudah ada (Beranda, Daftar/Tambah Buku, Daftar/Tambah Anggota – Jobsheet 1-3) belum mencakup fitur Login, Dashboard Petugas, dan Peminjaman/Pengembalian. Dokumen ini merancang wireframe untuk halaman-halaman tersebut sebelum diimplementasikan mulai Jobsheet 5 dan seterusnya.

Aktor

- **Tamu:** hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
- **Petugas:** login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

User Flow – Peminjaman Buku

```
[Petugas Login] -> [Dashboard] -> [Pilih menu "Peminjaman Baru"]  
-> [Pilih Anggota] -> [Pilih Buku (stok > 0)]  
-> [Simpan] -> [Stok buku berkurang 1] -> [Kembali ke Dashboard]
```

User Flow – Pengembalian Buku

```
[Dashboard] -> [Menu "Pengembalian"] -> [Cari transaksi aktif (anggota/buku)]  
-> [Tandai "Dikembalikan"] -> [Stok buku bertambah 1]  
-> [Kembali ke Dashboard]
```

Wireframe: Halaman Login

```
+-----+  
|          SIMPUS-Mini          |  
+-----+  
|                                |  
|      [ Login Petugas ]       |  
|                                |  
| Username : [_____]           |  
| Password : [_____]           |  
|                                |  
|      [   Masuk   ]           |  
|                                |  
+-----+
```

| Belum punya akun? Daftar di sini |
+-----+

Wireframe: Dashboard Petugas

```
+-----+
| SIMPUS-Mini Beranda | Buku | Anggota | Peminjaman | (Nama Petugas) Logout |
|-----|
| [Total Buku] [Total Anggota] [Sedang Dipinjam] |
| |
| Aksi Cepat: |
| [ + Peminjaman Baru ] [ + Pengembalian ] |
| |
| Transaksi Terbaru |
| ----- |
| Anggota | Buku | Tgl Pinjam | Status |
+-----+
```

Wireframe: Form Peminjaman

```
+-----+
| Form Peminjaman Buku |
|-----|
| Anggota : [ dropdown pilih anggota ]|
| Buku : [ dropdown, hanya stok>0 ]|
| Tanggal Pinjam : [ auto: hari ini ] |
| |
| [ Simpan Peminjaman ] |
+-----+
```

Wireframe: Form Pengembalian

```
+-----+
| Pengembalian Buku |
|-----|
| Cari transaksi aktif: |
| [ nama anggota / judul buku _____ ]|
```

```
| |
| Anggota | Buku | Tgl Pinjam | [Kembalikan] |
+-----+
```

Wireframe: Riwayat Peminjaman per Anggota

```
+-----+
| Riwayat Peminjaman – Siti Aminah |
|-----|
| Buku          | Pinjam    | Kembali | Status |
| Laskar Pelangi | 01/07     | 10/07   | Selesai |
| Bumi Manusia  | 15/07     | -       | Dipinjam |
+-----+
```

Konsistensi dengan Desain yang Sudah Berjalan

- Warna aksen, tipografi navbar, dan gaya tabel/kartu mengikuti `assets/css/style.css` yang sudah dibangun sejak Jobsheet 2-3.
- Navbar akan ditambah menu **Peminjaman** dan indikator status login (nama petugas / tombol Logout) mulai implementasi di Jobsheet 10.
- Edge case yang perlu ditangani saat implementasi: buku stok habis tidak boleh dipilih di form peminjaman; anggota dengan tunggakan terlambat divalidasi di Jobsheet 12 (tugas mandiri).

Dokumentasi Jobsheet 5 – JavaScript DOM & Event

Dokumentasi ini melanjutkan [dokumentasi jobsheet-03](#) (HTML/CSS responsif) dan [jobsheet-04](#) (rancangan UI/UX). Jobsheet-05 adalah titik penting: ini **pertama** kalinya** aplikasi SIMPUS-Mini punya **JavaScript** – kode yang membuat** halaman benar-benar bisa "bereaksi" terhadap tindakan pengguna, bukan sekadar tampilan statis.

Tentang `docs/wireframe.md`

File ini **identik persis** dengan [docs/wireframe.md](#) [di jobsheet-04](#) — tidak ada perubahan rancangan UI/UX baru di jobsheet ini. Kalau kamu belum membaca rancangan itu, baca dulu [dokumentasi jobsheet-04](#) sebelum lanjut ke sini.

Apa yang Baru di Jobsheet 5?

Sesuai [README.md](#) jobsheet ini, ada 4 penambahan besar, semuanya lewat file baru `assets/js/app.js`:

1. **Menu hamburger diganti dari CSS ke JavaScript** — sebelumnya memakai "checkbox hack" murni CSS (lihat [dokumentasi jobsheet-03](#)), sekarang memakai tombol asli + `classList.toggle()`.
2. **Validasi form sisi klien (client-side)** — form Tambah Buku dan Tambah Anggota sekarang menolak data yang tidak valid **sebelum** ter-submit, dengan pesan error yang muncul langsung di halaman.
3. **Filter/pencarian tabel real-time** — mengetik di kolom cari langsung menyaring baris tabel tanpa reload halaman.
4. **Tombol Hapus yang benar-benar berfungsi** (di sisi tampilan) — menampilkan konfirmasi lalu menghapus barisnya dari layar.

Daftar Isi

1. [Konsep Dasar JavaScript & DOM](#)
2. [Apa yang Berubah di File HTML?](#)
3. [CSS Pendukung Fitur JavaScript](#)
4. [JS: Menu Hamburger](#)
5. [JS: Konfirmasi Hapus](#)
6. [JS: Filter Tabel Real-Time](#)
7. [JS: Validasi Form](#)
8. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-05/
├── index.html
├── assets/
│   ├── css/style.css      # Ditambah gaya untuk error & search-box
│   └── js/
```

```
|      └─ app.js          # BARU – seluruh interaktivitas jobsheet ini
|      └─ buku/
|          └─ list.html    # Ditambah kolom pencarian + class btn-hapus
|          └─ tambah.html  # Ditambah id="form-tambah" untuk validasi
|      └─ anggota/
|          └─ list.html
|          └─ tambah.html
|      └─ docs/wireframe.md # Identik dengan jobsheet-04
|      └─ README.md
|      └─ Dokumentasi/     # Folder dokumentasi ini
```

Catatan penting dari [README.md](#) jobsheet ini yang perlu diingat sejak awal: validasi di jobsheet ini murni berjalan di **browser pengguna** (client-side) dan **bisa dilewati** kalau JavaScript dinonaktifkan — belum aman untuk diandalkan sepenuhnya. Validasi sisi server (server-side) yang wajib dan tidak bisa dilewati baru akan ditambahkan di Jobsheet 7. Begitu juga tombol Hapus: baru menghilangkan baris dari tampilan, **belum** benar-benar menghapus data — itu menyusul di Jobsheet 9.

1. Konsep Dasar JavaScript & DOM

Ini pengenalan pertamamu ke JavaScript di seluruh rangkaian jobsheet ini. Kenali dulu istilah-istilah dasarnya sebelum masuk ke kode `app.js`.

1.1 Apa Beda HTML, CSS, dan JavaScript?

Kamu sudah tahu HTML mengatur **struktur/isi** ([dokumentasi jobsheet-01](#)) dan CSS mengatur **tampilan** ([dokumentasi jobsheet-02](#)). **JavaScript (JS)** menambah lapisan ketiga: **perilaku/interaktivitas** — apa yang terjadi ketika pengguna mengklik sesuatu, mengetik sesuatu, atau men-submit form. Tiga lapisan ini sengaja dipisah ke file berbeda (`.html`, `.css`, `.js`) supaya masing-masing bisa diubah tanpa mengganggu yang lain — prinsip yang sama dengan *separation of concerns* yang sudah dibahas di [dokumentasi jobsheet-02 §2.2](#).

1.2 Menghubungkan JavaScript ke HTML

```
<script src="assets/js/app.js"></script>
```

Bandungkan dengan cara menghubungkan CSS (`<link rel="stylesheet" href="...">`, lihat [dokumentasi jobsheet-02 §1.3](#)) — untuk JavaScript, tag yang dipakai adalah `<script>`, dengan atribut `src` menunjuk ke lokasi file `.js`-nya. Perhatikan di setiap halaman jobsheet ini, tag `<script>` diletakkan **di akhir** `<body>`, tepat sebelum `</body>` ditutup — bukan di `<head>` seperti `<link>` CSS. Alasannya dijelaskan di [§1.3](#).

1.3 Kenapa `<script>` Diletakkan di Akhir `<body>` ?

Browser membaca dan menjalankan file HTML **dari atas ke bawah, baris demi baris**. Kalau `<script>` diletakkan di `<head>` (di bagian paling** atas), kode JavaScript akan dijalankan **sebelum** elemen-elemen di `<body>` (seperti `<button id="nav-toggle-btn">` atau `<table>`) selesai dibuat oleh browser — akibatnya, kode JS yang mencoba mencari elemen itu akan gagal karena elemennya belum ada. Dengan meletakkan `<script>` di **paling akhir** `<body>`, seluruh HTML di atasnya sudah pasti selesai dibuat browser terlebih dulu sebelum `app.js` mulai dijalankan.

1.4 Apa itu DOM?

DOM (Document Object Model) adalah representasi halaman HTML dalam bentuk yang bisa "dibaca dan diubah" oleh JavaScript — bayangkan seluruh struktur HTML (setiap `<header>`, `<table>`, `<button>`, dst.) diubah menjadi sebuah "pohon" objek yang bisa dijelajahi dan dimodifikasi lewat kode. Sub-CPMK jobsheet ini secara eksplisit menyebut "manipulasi DOM & event" — DOM adalah **objek** yang dimanipulasi, sementara **event** (dibahas di [§1.6](#)) adalah **pemicu** yang menentukan kapan manipulasi itu terjadi.

1.5 Memilih Elemen dari DOM

`app.js` memakai 3 cara berbeda untuk "mengambil" elemen HTML supaya bisa dimanipulasi:

Fungsi	Mengambil	Contoh Pemakaian di <code>app.js</code>
<code>document.getElementById("id")</code>	Satu elemen berdasarkan atribut <code>id</code> -nya (harus unik di satu halaman — ingat aturan <code>id</code> unik dari dokumentasi jobsheet-01).	<code>document.getElementById("nav-toggle-btn")</code>

Fungsi	Mengambil	Contoh Pemakaian di <code>app.js</code>
<code>document.querySelector("selector")</code>	Elemen pertama yang cocok dengan selector CSS (boleh selector apa pun yang sudah kamu kenal dari dokumentasi jobsheet-02).	<code>document.querySelector("header nav")</code>
<code>document.querySelectorAll("selector")</code>	Semua elemen yang cocok dengan selector, sebagai kumpulan (bisa diulang satu-satu dengan <code>.forEach()</code>).	<code>document.querySelectorAll(".btn-hapus")</code>

Menariknya, `querySelector` / `querySelectorAll` memakai **selector CSS** yang sama persis** dengan yang kamu tulis di `style.css` — ini kabar** baik, artinya kemampuan membaca selector CSS yang sudah kamu kuasai sejak jobsheet-02 langsung berguna juga untuk menulis JavaScript.

1.6 Apa itu Event dan Event Listener?

Event adalah "kejadian" yang terjadi di halaman — pengguna mengklik (`click`), mengetik lalu melepas tombol keyboard (`keyup`), atau menekan tombol submit form (`submit`). **Event listener** adalah kode yang "menunggu" kejadian tertentu terjadi pada sebuah elemen, lalu menjalankan fungsi tertentu sebagai responsnya:

```
elemen.addEventListener("click", function () {
    // kode ini dijalankan SETIAP KALI elemen di-klik
});
```

Polanya selalu sama: **pilih elemen** (§1.5) → panggil `.addEventListener(namaEvent, fungsi)` di elemen itu → tulis fungsi yang berisi apa yang harus terjadi. Tiga event yang dipakai di jobsheet ini:

Event	Terjadi Saat	Dipakai di
click	Elemen diklik	Tombol hamburger (bab 4), tombol Hapus (bab 5)
keyup	Tombol keyboard dilepas (setelah diketik)	Kolom pencarian (bab 6)
submit	Form akan dikirim (tombol submit ditekan)	Validasi form (bab 7)

1.7 Struktur Umum Kode di `app.js`

Sebelum membedah tiap fitur satu per satu di bab 4-7, perhatikan pola besar yang diulang di seluruh file:

```
function initNavToggle() {
    // ambil elemen, pasang event listener
}

// ...fungsi init lain...

document.addEventListener("DOMContentLoaded", function () {
    initNavToggle();
    initHapusConfirm();
    initTableFilter();
    initValidasiForm();
});
```

- Setiap fitur dibungkus dalam **funksinya sendiri** (`initNavToggle`, `initHapusConfirm`, dst.) — kebiasaan menulis kode yang baik supaya tiap fitur mudah dibaca terpisah, bukan tercampur semua jadi satu blok kode panjang.
- `document.addEventListener("DOMContentLoaded", ...)` di baris paling bawah adalah "titik masuk" (entry point) seluruh script: `DOMContentLoaded` adalah event khusus yang terjadi **setelah seluruh** HTML selesai dimuat browser** menjadi DOM. Semua fungsi `init...` ** dipanggil di dalamnya untuk memastikan elemen yang mereka cari (misalnya `#nav-toggle-btn`) **sudah pasti ada** di DOM saat kode dijalankan — lapisan keamanan tambahan di atas penempatan `<script>` di akhir `<body>` yang sudah dibahas di [§1.3](#).
- Perhatikan juga tiap fungsi `init...` di `app.js` selalu diawali pengecekan seperti `if (!toggleBtn || !nav) return;` — ini **penjaga keamanan**** (*guard clause*): kalau elemen yang

dicari ternyata tidak** ada di halaman itu (misalnya halaman tanpa tabel tidak punya elemen `.table-responsive table`), fungsinya berhenti lebih awal (`return`) alih-alih error karena mencoba memanggil method pada sesuatu yang `null`. Berkat penjaga ini, satu file `app.js` yang sama bisa dipakai di **semua** halaman (Beranda tanpa tabel, halaman list dengan tabel, halaman tambah dengan form) tanpa menimbulkan error di halaman yang tidak punya elemen tertentu.

Dengan bekal istilah-istilah ini, sekarang kamu siap membaca perubahan konkretnya mulai bab 2.

Lanjut ke: [Apa yang Berubah di File HTML?](#)

2. Apa yang Berubah di File HTML?

Sebelum membedah `app.js` fungsi per fungsi, kenali dulu perubahan kecil di HTML yang menjadi "kait" (hook) bagi JavaScript untuk mencari elemen yang perlu dimanipulasi. Tanpa perubahan-perubahan ini, `app.js` tidak akan punya elemen yang bisa ditemukan lewat `getElementById / querySelector` (ingat konsepnya dari [bab 1 §1.5](#)).

2.1 Hamburger: dari Checkbox ke Tombol Asli

Sebelumnya (jobsheet-03), menu hamburger dibuat dari pasangan `<input type="checkbox"> + <label>` (checkbox hack, lihat [dokumentasi jobsheet-03](#)). **Sekarang:**

```
<button type="button" id="nav-toggle-btn" class="nav-toggle-label" aria-label="Menu">&#9776;</button>
```

- Elemen `<input type="checkbox">` **dihapus total** — tidak dibutuhkan lagi karena status buka/tutup menu sekarang disimpan lewat class CSS yang ditambahkan JavaScript, bukan status "tercentang" checkbox (dijelaskan detail di [bab 4](#)).
- `<label>` diganti jadi `<button type="button">` — tombol asli yang memang dirancang untuk diklik, dengan `id="nav-toggle-btn"` sebagai "kait" supaya `document.getElementById("nav-toggle-btn")` di `app.js` bisa menemukannya.
- Class `nav-toggle-label` **tetap dipertahankan** namanya (meski elemennya sudah bukan `<label>` lagi) supaya gaya CSS yang sudah ada (ukuran font, warna, dll. — lihat [dokumentasi jobsheet-03 §3.3](#)) tidak perlu ditulis ulang. Perubahan yang perlu di CSS dibahas di [bab 3](#).
- `aria-label="Menu"` adalah atribut baru untuk **aksesibilitas** — memberi tahu pembaca layar (*screen reader*) bahwa tombol berisi simbol ☰ ini fungsinya adalah "Menu", karena teks

☰ sendiri (ingat entity ini dari [dokumentasi jobsheet-03 §2.2](#)) tidak bermakna apa pun kalau dibacakan sebagai teks biasa.

2.2 Kolom Pencarian Baru di Halaman Daftar

Di `buku/list.html` dan `anggota/list.html`, ada `<div>` baru sebelum tabel:

```
<div class="search-box">
  <label for="search-input">Cari Judul Buku</label>
  <input type="text" id="search-input" placeholder="Ketik judul buku...">
</div>
```

- Pola `<label for="..."> + <input id="...">` ini **sama persis** dengan pola form yang sudah kamu kuasai sejak [dokumentasi jobsheet-01](#) — meskipun `<input>` ini **tidak berada di dalam** `<form>`, karena memang tidak dimaksudkan untuk "disimpan" atau di-submit, hanya untuk dibaca nilainya secara langsung oleh JavaScript setiap kali diketik (dijelaskan di [bab 6](#)).
- Atribut `placeholder` adalah atribut HTML yang belum pernah dipakai di jobsheet-jobsheet sebelumnya — menampilkan teks abu-abu contoh (misalnya "Ketik judul buku...") di dalam kotak input **selama masih** kosong**, teks itu otomatis hilang begitu pengguna mulai mengetik.** Beda dengan `value`, teks placeholder **tidak ikut terkirim** kalau form di-submit.
- `id="search-input"` adalah "kait" yang dicari `document.getElementById("search-input")` di `app.js` ([bab 6](#)).

2.3 Class `btn-hapus` di Tombol Hapus

```
<button type="button">Edit</button>
<button type="button" class="btn-hapus">Hapus</button>
```

Ingat dari [dokumentasi jobsheet-01](#), kedua tombol Edit dan Hapus di kolom "Aksi" sejak awal memang belum berfungsi (`type="button"` tanpa aksi apa pun). Sekarang tombol Hapus diberi `class="btn-hapus"` — bukan untuk keperluan gaya CSS (warnanya tetap diatur lewat `:last-of-type` seperti dijelaskan di [dokumentasi jobsheet-02 §7.6](#)), melainkan supaya `document.querySelectorAll(".btn-hapus")` di `app.js` bisa menemukan **semua** tombol Hapus sekaligus di satu halaman (dijelaskan di [bab 5](#)). Tombol **Edit** **tidak** diberi `class`** apa pun karena jobsheet ini memang belum menambahkan** fungsi apa pun untuknya.

2.4 `id="form-tambah"` di Form Tambah Buku/Anggota

```
<form id="form-tambah">
```

Kedua form (buku/tambah.html dan anggota/tambah.html) sekarang punya id="form-tambah" di tag <form>-nya — sebelumnya ([dokumentasi jobsheet-01](#)) tag <form> tidak punya atribut apa pun. id ini adalah "kait" yang dicari document.getElementById("form-tambah") supaya JavaScript bisa memasang **event listener submit** untuk validasi (dijelaskan detail di [bab 7](#)). Perhatikan **kedua form** memakai id yang **sama** (form-tambah) meskipun field-fieldnya berbeda (Judul/ Pengarang untuk buku, Nama/No. Anggota untuk anggota) — ini aman karena id hanya perlu unik **di dalam satu halaman**, dan kedua form ini berada di halaman yang berbeda. Cara app.js menangani perbedaan field di kedua form ini dengan **satu fungsi validasi yang sama** dibahas di [bab 7](#).

Lanjut ke: [CSS Pendukung Fitur JavaScript](#)

3. CSS Pendukung Fitur JavaScript

Beberapa baris style.css berubah atau ditambah, semuanya untuk mendukung fitur JavaScript baru di [bab 4](#) sampai [bab 7](#). Bab ini membahas perubahan CSS itu **sebelum** masuk ke JavaScript-nya sendiri, supaya alurnya runtut.

3.1 Hamburger: dari .nav-toggle ke Tombol Asli

Dihapus dari style.css:

```
.nav-toggle {  
  display: none;  
}
```

Aturan ini dulu menyembunyikan elemen <input type="checkbox"> (lihat [dokumentasi jobsheet-03 §3.2](#)). Karena elemen checkbox-nya sendiri sudah dihapus dari HTML ([bab 2 §2.1](#)), aturan CSS untuk menyembunyikannya juga tidak diperlukan lagi.

Ditambah ke .nav-toggle-label (yang sekarang menata elemen <button>, bukan <label>):

```
.nav-toggle-label {  
  display: none;  
  font-size: 1.6rem;  
  color: #fff;  
  background: none;
```

```
border: none;
cursor: pointer;
}
```

Dua baris baru, `background: none;` dan `border: none;`, **tidak** diperlukan** saat elemennya masih `<label>` (label tidak punya latar** atau bingkai bawaan), tapi **wajib** sekarang karena elemennya adalah `<button>` — dan tombol HTML secara default punya latar abu-abu dan bingkai 3D bawaan browser. Tanpa dua baris ini, tombol hamburger akan terlihat seperti kotak abu-abu biasa, bukan ikon `☰` polos yang menyatu dengan warna header biru.

Berubah di dalam `@media (max-width: 480px):`

```
header nav.nav-open {
  display: block;
}
```

Sebelumnya (jobsheet-03): `.nav-toggle:checked ~ nav { display: block; }` — selector berbasis **status checkbox** (`:checked`) dan **sibling combinator**** `~` (ingat konsepnya dari** [dokumentasi jobsheet-03 §3.5](#)). Sekarang selector-nya jauh lebih sederhana: `header nav.nav-open` — elemen `<nav>` di dalam `<header>` yang **punya class** `nav-open`. Tidak ada lagi pseudo-class atau sibling combinator sama sekali, karena status "menu terbuka" sekarang murni ditentukan oleh **ada atau tidaknya**** class `nav-open` — dan yang menambah/menghapus class itu** adalah JavaScript (dijelaskan di [bab 4](#)), bukan lagi status tercentang sebuah checkbox tersembunyi.

3.2 Gaya Baru: Pesan Error Validasi

```
/* ===== Pesan Error Validasi ===== */
.error {
  display: block;
  color: #d9534f;
  font-size: 0.85rem;
  margin-top: 0.25rem;
}
```

Class `error` ini **belum ada** di HTML manapun secara statis — class ini akan **dibuat dan disisipkan sepenuhnya oleh JavaScript** setiap kali validasi form gagal (dijelaskan detail di [bab 7](#)). CSS ini menyiapkan **bagaimana rupanya** kalau/ketika elemen `<span class="error">` itu benar-benar muncul nanti:

- `display: block;` — memastikan pesan error tampil di **baris baru** sendiri, di bawah kotak input, bukan menempel sejajar di sampingnya (ingat elemen `<span>` secara default bersifat

inline, sama seperti pembahasan `display: block` pada `<label>` di [dokumentasi jobsheet-02 §8.3](#)).

- `color: #d9534f;` — warna merah, sama persis dengan warna tombol Hapus ([dokumentasi jobsheet-02 §7.6](#)) — konsisten menandakan "sesuatu yang perlu perhatian/tindakan" di seluruh aplikasi.
- `font-size: 0.85rem;` dan `margin-top: 0.25rem;` — teks sedikit lebih kecil dari input di atasnya, dengan jarak tipis supaya terlihat jelas sebagai keterangan tambahan, bukan menyatu dengan input.

3.3 Gaya Baru: Kolom Pencarian

```
/* ===== Kolom Pencarian ===== */
.search-box {
  margin-bottom: 1rem;
}

.search-box input {
  width: 100%;
  max-width: 320px;
  padding: 0.5rem 0.75rem;
  border: 1px solid #cdd4da;
  border-radius: 4px;
}
```

- `.search-box { margin-bottom: 1rem; }` memberi jarak antara kotak pencarian dan tabel di bawahnya.
- `.search-box input` menata kotak input pencarian mirip gaya input form yang sudah kamu kenal dari [dokumentasi jobsheet-02 §8.4](#) — border tipis, sudut membulat, padding nyaman — hanya saja `max-width` dibuat lebih sempit (`320px` , dibanding `400px` pada input form) karena kolom pencarian memang tidak perlu selebar field form biasa.

Dengan CSS sudah siap menyambut elemen-elemen baru ini, sekarang saatnya membedakan JavaScript yang benar-benar menggerakkannya, dimulai dari menu hamburger.

Lanjut ke: [JS: Menu Hamburger](#)

4. JS: Menu Hamburger

Fungsi pertama di `app.js`, dan yang paling sederhana — cocok jadi titik awal belajar membaca kode JavaScript.

4.1 Kode Lengkap

```
// ===== Hamburger menu (JS-driven, menggantikan checkbox hack) =====  
function initNavToggle() {  
    const toggleBtn = document.getElementById("nav-toggle-btn");  
    const nav = document.querySelector("header nav");  
    if (!toggleBtn || !nav) return;  
  
    toggleBtn.addEventListener("click", function () {  
        nav.classList.toggle("nav-open");  
    });  
}
```

4.2 Mengambil Dua Elemen yang Dibutuhkan

```
const toggleBtn = document.getElementById("nav-toggle-btn");  
const nav = document.querySelector("header nav");
```

- `const` adalah kata kunci untuk mendeklarasikan sebuah **variabel** (tempat menyimpan nilai) yang nilainya **tidak akan diganti** lagi setelah didefinisikan — cocok dipakai di sini karena `toggleBtn` dan `nav` selalu merujuk ke elemen yang sama sepanjang fungsi ini berjalan.
- `toggleBtn` diisi dengan tombol hamburger (`id="nav-toggle-btn"` , ingat dari [bab 2 §2.1](#)).
- `nav` diisi dengan elemen `<nav>` yang berada di dalam `<header>` — memakai selector CSS `"header nav"` yang **persis sama** dengan descendant selector yang sudah kamu kenal dari [dokumentasi jobsheet-02](#).

4.3 Penjaga Keamanan (Guard Clause)

```
if (!toggleBtn || !nav) return;
```

- Tanda seru `!` di depan sebuah nilai berarti **kebalikan/negasi** — `!toggleBtn` bernilai benar (`true`) kalau `toggleBtn` adalah `null` (tidak ditemukan elemennya).
- `||` berarti **atau** — kondisi ini benar kalau **salah satu saja** dari `toggleBtn` atau `nav` tidak ditemukan.
- Kalau kondisi ini benar, `return;` menghentikan fungsi **sebelum** baris berikutnya (`toggleBtn.addEventListener(...)`) sempat dijalankan — mencegah error "Cannot read

properties of null" yang akan muncul kalau kita mencoba memanggil `.addEventListener` pada nilai `null`.

Baris ini persis konsep yang sudah disinggung di [dokumentasi jobsheet-05 §1.7](#): karena `app.js` yang sama dimuat di **semua** halaman lewat `<script src="assets/js/app.js">`, dan fungsi ini dipanggil di semua halaman itu juga (lihat [bab 1 §1.7](#)), guard clause ini memastikan kode tidak akan error meskipun (secara hipotetis) suatu halaman ternyata tidak punya tombol hamburger.

4.4 Memasang Event Listener

```
toggleBtn.addEventListener("click", function () {  
  nav.classList.toggle("nav-open");  
});
```

- `toggleBtn.addEventListener("click", ...)` — ingat pola ini dari [bab 1 §1.6](#): "setiap kali tombol ini diklik, jalankan fungsi berikut."
- `nav.classList` adalah objek yang mewakili **daftar semua class** yang dimiliki elemen `nav` saat ini (mirip atribut `class="..."` di HTML, tapi dalam bentuk yang bisa diprogram).
- `.toggle("nav-open")` adalah method yang **membalik status** satu class tertentu:
 - Kalau elemen `nav` **belum** punya class `nav-open` → class itu **ditambahkan**.
 - Kalau elemen `nav` **sudah** punya class `nav-open` → class itu **dihapus**.

Iniilah kenapa satu fungsi klik yang sama bisa dipakai untuk **membuka** dan menutup** menu secara bergantian — kita tidak perlu menulis kondisi** `if/else` manual untuk mengecek status sebelumnya, `classList.toggle()` sudah menangani logika bolak-balik itu secara otomatis.

4.5 Menghubungkan Kembali ke CSS

Ingat dari [bab 3 §3.1](#), CSS di dalam `@media (max-width: 480px)` punya aturan:

```
header nav.nav-open {  
  display: block;  
}
```

Alur lengkapnya sekarang:

1. Pengguna mengklik tombol hamburger (`#nav-toggle-btn`).
2. Event listener `click` di [§4.4](#) terpicu.
3. `nav.classList.toggle("nav-open")` menambahkan class `nav-open` ke elemen `<nav>`.

4. CSS `header nav.nav-open { display: block; }` otomatis berlaku karena elemen `<nav>` sekarang cocok dengan selector itu → menu **muncul**.
5. Klik tombol sekali lagi → `classList.toggle()` **menghapus** class `nav-open` → elemen `<nav>` tidak lagi cocok dengan selector CSS tadi → menu kembali ke `display: none` dari gaya dasarnya → menu **tersembunyi lagi**.

4.6 Bandingkan dengan Checkbox Hack (Jobsheet-03)

	Checkbox Hack (Jobsheet-03)	JS-Driven (Jobsheet-05)
Penyimpan status	Atribut <code>checked</code> pada <code><input type="checkbox"></code> tersembunyi	Class <code>nav-open</code> pada elemen <code><nav></code>
Yang mengubah status	Browser secara otomatis (perilaku bawaan checkbox)	Kode JavaScript (<code>classList.toggle</code>)
Selector CSS	<code>.nav-toggle:checked ~ nav</code> (pseudo-class + sibling combinator)	<code>header nav.nav-open</code> (selector class biasa)
Butuh JavaScript?	Tidak sama sekali	Ya

Checkbox hack ([dokumentasi jobsheet-03](#)) tetap merupakan trik CSS yang sah dan berguna terutama saat JavaScript belum dipelajari/tersedia. Tapi begitu JavaScript masuk (seperti di jobsheet ini), pendekatan berbasis `classList` biasanya **lebih mudah** dibaca dan dikembangkan** – tidak perlu memahami sibling combinator** untuk sekadar tahu "menu ini sedang terbuka atau tertutup", cukup periksa ada-tidaknya satu class.

Lanjut ke: [JS: Konfirmasi Hapus](#)

5. JS: Konfirmasi Hapus

Fungsi ini membuat tombol "Hapus" di tabel Daftar Buku/Anggota – yang sejak jobsheet-01 hanya pajangan tanpa fungsi ([dokumentasi jobsheet-01](#)) – akhirnya benar-benar melakukan sesuatu.

5.1 Kode Lengkap

```
// ===== Konfirmasi hapus (front-end only, belum ke server) =====
function initHapusConfirm() {
  document.querySelectorAll(".btn-hapus").forEach(function (btn) {
    btn.addEventListener("click", function () {
      const row = btn.closest("tr");
      const nama = row ? row.querySelector("td")?.textContent : "data ini";
      const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
      if (yakin && row) {
        row.remove();
      }
    });
  });
}
```

5.2 Memasang Event Listener ke Banyak Tombol Sekaligus

```
document.querySelectorAll(".btn-hapus").forEach(function (btn) {
  btn.addEventListener("click", function () {
    // ...
  });
});
```

Ingat dari [bab 1 §1.5](#), `querySelectorAll(".btn-hapus")` mengambil **semua** tombol Hapus di satu halaman (5 tombol di Daftar Buku, 2 tombol di Daftar Anggota — ingat class ini baru ditambahkan di [bab 2 §2.3](#)). Karena hasilnya berupa **kumpulan** elemen (bukan satu elemen seperti `getElementById`), kita perlu `.forEach(...)` untuk mengulang satu per satu, dan memasang `addEventListener` **terpisah** ke tiap tombol — setiap tombol Hapus jadi punya "pendengar klik"-nya sendiri-sendiri.

5.3 Mencari Baris Tabel yang Jadi Induk Tombol

```
const row = btn.closest("tr");
```

`.closest("tr")` adalah method yang mencari **ke atas** dari elemen `btn` (tombol yang diklik) menuju elemen induknya, berhenti begitu menemukan elemen pertama yang cocok dengan selector `"tr"` — dalam kasus ini, baris tabel (`<tr>`) yang membungkus tombol tersebut (ingat struktur tabel dari [dokumentasi jobsheet-01 §3.2](#): tombol Hapus berada di dalam `<td>`, yang berada di dalam

`<tr>).closest()` berguna justru karena tombolnya sendiri **tidak tahu** baris seberapa dirinya berada — ia hanya tahu "cari `<tr>` terdekat di atasku", tidak peduli itu baris ke-1, ke-3, atau ke-5.

5.4 Mengambil Nama/Judul dari Baris Itu

```
const nama = row ? row.querySelector("td)?.textContent : "data ini";
```

Baris ini terlihat rumit, mari dibedah dari luar ke dalam:

- kondisi `? nilaiJikaBenar : nilaiJikaSalah` adalah **ternary operator**** — bentuk singkat dari `if/else` yang ditulis dalam satu** baris sebagai sebuah *nilai* (bukan sebagai blok kode terpisah). Baris ini setara dengan:

```
let nama;
if (row) {
  nama = row.querySelector("td)?.textContent;
} else {
  nama = "data ini";
}
```

- `row.querySelector("td")` — mengambil sel `<td>` **pertama** di dalam baris itu (ingat urutan kolom Daftar Buku: Judul, Pengarang, Tahun, Stok, Aksi — jadi `<td>` pertama selalu berisi **Judul** buku, atau **No. Anggota** di tabel Anggota).
- `?.` (disebut *optional chaining*) — mirip titik (`.`) biasa untuk mengakses properti, tapi **aman** kalau nilai di depannya `null` / `undefined`. Kalau `row.querySelector("td")` ternyata tidak menemukan apa-apa (mengembalikan `null`), `?.textContent` tidak akan menyebabkan error, melainkan mengembalikan `undefined` dengan aman.
- `.textContent` — mengambil **teks yang tampil** di dalam elemen itu (misalnya `"Laskar Pelangi"`).
- Kalau `row` sendiri ternyata `null` (kasus yang seharusnya jarang terjadi), fallback teks `"data ini"` dipakai sebagai gantinya, supaya pesan konfirmasi tetap masuk akal (lihat [§5.5](#)).

5.5 Menampilkan Dialog Konfirmasi

```
const yakin = confirm("Yakin ingin menghapus \"" + nama + "\"?");
```

`confirm(pesan)` adalah fungsi bawaan browser (bukan sesuatu yang perlu diimpor/didefinisikan) yang menampilkan **kotak dialog bawaan browser**** berisi pesan, dengan dua tombol: **OK** dan **Cancel**. Fungsi** ini **menghentikan sementara** eksekusi kode (dan interaksi pengguna dengan

halaman) sampai pengguna memilih salah satu tombol, lalu mengembalikan nilai `true` (kalau OK diklik) atau `false` (kalau Cancel diklik) — nilai itulah yang disimpan ke variabel `yakin`.

Perhatikan tanda kutip dua yang di-*escape* dengan backslash (`\`) di dalam string yang juga diapit tanda kutip dua — ini diperlukan supaya JavaScript tidak salah mengira tanda kutip di tengah teks sebagai akhir dari string. Hasil akhirnya, pesan yang muncul ke pengguna misalnya:

Yakin ingin menghapus "Laskar Pelangi"?

5.6 Menghapus Baris dari Tampilan

```
if (yakin && row) {  
  row.remove();  
}
```

- `&&` berarti **dan** — kode di dalam `if` hanya dijalankan kalau **kedua** kondisi benar: pengguna menekan OK (`yakin` bernilai `true`) **dan** baris `row` memang ditemukan.
- `row.remove()` adalah method DOM yang menghapus elemen itu **dari** tampilan halaman** secara langsung — baris tabel itu akan lenyap dari** layar seketika, tanpa perlu me-reload halaman.

Penting untuk diingat (sudah disinggung juga di [README.md](#) jobsheet ini): `row.remove()` **hanya** menghapus elemen dari DOM/tampilan browser saat ini. Begitu halaman di-refresh, baris itu akan **muncul kembali** karena datanya masih tertulis apa adanya di file HTML — belum ada mekanisme yang benar-benar menghapus data secara permanen (misalnya ke database). Fitur hapus yang sungguhan baru akan dibangun mulai Jobsheet 9.

Lanjut ke: [JS: Filter Tabel Real-Time](#)

6. JS: Filter Tabel Real-Time

Fungsi ini menghubungkan kolom pencarian baru ([bab 2 §2.2](#)) dengan tabel di bawahnya, menyaring baris **sambil** pengguna mengetik — tanpa tombol "Cari" terpisah, tanpa reload halaman.

6.1 Kode Lengkap

```
// ===== Filter/pencarian tabel real-time =====
function initTableFilter() {
  const input = document.getElementById("search-input");
  const table = document.querySelector(".table-responsive table");
  if (!input || !table) return;

  input.addEventListener("keyup", function () {
    const keyword = input.value.toLowerCase();
    const rows = table.querySelectorAll("tbody tr");
    rows.forEach(function (row) {
      const teks = row.textContent.toLowerCase();
      row.style.display = teks.includes(keyword) ? "" : "none";
    });
  });
}
```

6.2 Mengambil Kotak Input dan Tabelnya

```
const input = document.getElementById("search-input");
const table = document.querySelector(".table-responsive table");
```

- `input` — kotak pencarian, dicari lewat `id="search-input"` (ingat dari [bab 2 §2.2](#)).
- `table` — dicari dengan selector `".table-responsive table"`, descendant selector yang mengambil elemen `<table>` di dalam `<div class="table-responsive">` (ingat wrapper ini dari [dokumentasi jobsheet-03](#)). Selector ini sengaja **spesifik** (bukan cuma `"table"`) supaya kalau suatu saat ada tabel lain di halaman yang sama untuk keperluan berbeda, fungsi ini tidak salah sasaran.
- Guard clause `if (!input || !table) return;` — pola yang sama persis dengan [bab 4 §4.3](#), memastikan fungsi ini aman dipanggil di halaman mana pun (termasuk Beranda yang tidak punya kolom pencarian maupun tabel sama sekali).

6.3 Event `keyup`: Bereaksi Setiap Ketikan

```
input.addEventListener("keyup", function () {
  // ...
});
```

Ingat dari [bab 1 §1.6](#), event `keyup` terjadi setiap kali sebuah tombol keyboard **dilepas** (setelah ditekan) saat fokus berada di elemen `input`. Karena event ini terpicu untuk **setiap huruf** yang

diketik (bukan menunggu pengguna menekan Enter atau tombol "Cari"), efeknya terasa seperti pencarian "real-time" — tabel langsung tersaring seiring pengguna mengetik.

6.4 Mengambil Kata Kunci Pencarian

```
const keyword = input.value.toLowerCase();
```

- `input.value` — nilai/teks yang **sedang** diketik di dalam kotak input saat ini (berbeda dengan placeholder dari [bab 2 §2.2](#) yang hanya teks contoh, bukan nilai sungguhan).
- `.toLowerCase()` — mengubah semua huruf jadi huruf kecil. Ini penting supaya pencarian **tidak peka huruf besar/kecil** (*case-insensitive*) — mengetik "laskar" tetap menemukan "Laskar Pelangi" meskipun huruf "L"-nya besar di data aslinya.

6.5 Mengulang Setiap Baris Tabel

```
const rows = table.querySelectorAll("tbody tr");
rows.forEach(function (row) {
  const teks = row.textContent.toLowerCase();
  row.style.display = teks.includes(keyword) ? "" : "none";
});
```

- `table.querySelectorAll("tbody tr")` — mengambil **semua** baris data (ingat dari [dokumentasi jobsheet-01 §3.2](#), baris data berada di dalam `<tbody>`, terpisah dari baris judul kolom di `<thead>` — sehingga baris judul **tidak ikut** disaring/ disembunyikan).
- Untuk **setiap** baris, `row.textContent` mengambil **seluruh teks** di dalam baris itu (gabungan semua sel `<td>`-nya jadi satu string panjang, termasuk teks tombol "Edit"/"Hapus" di dalamnya), lalu diubah ke huruf kecil juga (`.toLowerCase()`) supaya konsisten dengan `keyword`.
- `teks.includes(keyword)` — mengembalikan `true` kalau teks **mengandung** `keyword` di bagian mana pun (bukan harus sama persis dari awal). Contoh: "laskar pelangi andrea hirata 2005 4" `.includes("hirata")` bernilai `true`.
- `row.style.display = ... ? "" : "none";` — mengatur properti CSS `display` **langsung dari JavaScript**:
 - Kalau `teks.includes(keyword)` bernilai `true` (baris cocok) → `row.style.display = ""` — mengosongkan nilai `display` supaya kembali ke perilaku default tabel (baris tampil normal).
 - Kalau `false` (tidak cocok) → `row.style.display = "none"` — baris **disembunyikan** total dari tampilan.

6.6 Kenapa Baris Disembunyikan, Bukan Dihapus?

Perhatikan fungsi ini memakai `row.style.display = "none"`, **bukan** `row.remove()` seperti pada tombol Hapus di [bab 5 §5.6](#). Ini perbedaan penting: `.remove()` **menghapus permanen** elemen dari DOM (perlu dibuat ulang untuk memunculkannya lagi), sedangkan `style.display = "none"` hanya **menyembunyikan sementara** — elemennya tetap ada di DOM, hanya tidak terlihat. Ini pilihan yang tepat untuk fitur pencarian: begitu pengguna **menghapus** teks di kolom cari (kembali kosong), fungsi ini berjalan lagi dan `keyword` jadi string kosong ("") — dan **setiap** teks baris pasti "mengandung" string kosong (`teks.includes("")` selalu `true`), sehingga **semua baris** otomatis muncul kembali** tanpa perlu logika tambahan apa pun.**

Lanjut ke: [JS: Validasi Form](#)

7. JS: Validasi Form

Bab paling panjang di jobsheet ini — validasi form melibatkan 3 fungsi yang saling bekerja sama: dua fungsi pembantu (`tampilkanError` , `hapusError`) dan satu fungsi utama (`initValidasiForm`).

7.1 Kode Lengkap

```
// ===== Validasi form (client-side) =====
function tampilkanError(input, pesan) {
    hapusError(input);
    const span = document.createElement("span");
    span.className = "error";
    span.textContent = pesan;
    input.insertAdjacentElement("afterend", span);
}

function hapusError(input) {
    const next = input.nextElementSibling;
    if (next && next.classList.contains("error")) {
        next.remove();
    }
}

function initValidasiForm() {
    const form = document.getElementById("form-tambah");
    if (!form) return;
```

```

form.addEventListener("submit", function (e) {
  let valid = true;

  const judul = form.querySelector("[name='judul']", [name='nama']");
  if (judul && judul.value.trim() === "") {
    tampilkanError(judul, "Field ini wajib diisi.");
    valid = false;
  } else if (judul) {
    hapusError(judul);
  }

  // ...(pengecekan pengarang, tahun, stok dengan pola serupa)...

  if (!valid) {
    e.preventDefault();
  }
});
}

```

7.2 Mengingat Dulu: Ingat Konsep "Belum Diproses" dari Jobsheet-01

Ingat dari [dokumentasi jobsheet-01 §4.2](#), tag `<form>` di halaman Tambah Buku/Anggota **tidak** punya atribut `action/method` — artinya menekan "Simpan" hanya me-reload halaman tanpa mengirim data ke mana pun. Validasi di jobsheet ini **tidak** mengubah fakta itu** — form masih belum benar-benar menyimpan data ke** mana pun. Yang baru ditambahkan hanyalah **pemeriksaan** yang berjalan tepat **sebelum** proses submit (yang sebenarnya belum melakukan apa-apa itu) — mempersiapkan pola yang nantinya tetap berguna begitu form sungguhan terhubung ke server.

7.3 Fungsi Pembantu: `tampilkanError`

```

function tampilkanError(input, pesan) {
  hapusError(input);
  const span = document.createElement("span");
  span.className = "error";
  span.textContent = pesan;
  input.insertAdjacentElement("afterend", span);
}

```

Fungsi ini menerima 2 **parameter** (nilai masukan): `input` (elemen field yang bermasalah) dan `pesan` (teks error yang ingin ditampilkan).

1. `hapusError(input);` — memanggil fungsi lain ([§7.4](#)) untuk **membersihkan dulu** pesan error lama (kalau ada) sebelum menambah yang baru — mencegah dua pesan error menumpuk untuk field yang sama.
2. `document.createElement("span")` — method DOM untuk **membuat** elemen HTML baru** dari kode JavaScript, sepenuhnya belum ada di** HTML manapun sampai baris ini dijalankan. Ini beda dengan seluruh fungsi sebelumnya (`initNavToggle`, `initHapusConfirm`, `initTableFilter`) yang hanya **memilih** elemen yang sudah ada — ini pertama kalinya `app.js` benar-benar **membuat** elemen baru.
3. `span.className = "error";` — memberi elemen `<span>` yang baru dibuat itu atribut `class="error"`, menghubungkannya ke gaya CSS `.error` yang sudah dibahas di [bab 3 §3.2](#).
4. `span.textContent = pesan;` — mengisi teks di dalam `<span>` itu dengan pesan error yang diberikan (misalnya "Field ini wajib diisi").
5. `input.insertAdjacentElement("afterend", span);` — method DOM untuk **menyisipkan** elemen baru ke posisi tertentu **relatif** terhadap elemen lain, tanpa perlu tahu struktur DOM di sekelilingnya secara detail. "afterend" berarti "tepat setelah elemen `input`, sebagai saudara seajar (sibling), bukan di dalamnya." Hasilnya, `<span class="error">` akan muncul **tepat di bawah** kotak input yang bermasalah (ingat `.error { display: block; }` dari [bab 3 §3.2](#) membuatnya pindah ke baris baru).

7.4 Fungsi Pembantu: `hapusError`

```
function hapusError(input) {  
  const next = input.nextElementSibling;  
  if (next && next.classList.contains("error")) {  
    next.remove();  
  }  
}
```

- `input.nextElementSibling` — properti yang mengambil elemen **tepat setelah** `input` di level yang sama (sibling berikutnya) — kebalikan logis dari `insertAdjacentElement("afterend", ...)` yang dipakai di [§7.3](#) untuk menyisipkannya.
- `next.classList.contains("error")` — memeriksa **apakah** elemen sibling berikutnya itu punya class `error` (ingat `classList` dari [bab 4 §4.4](#), `.contains()` di sini mengecek keberadaan satu class tertentu, berbeda dari `.toggle()` yang membalik status).
- Kalau memang ada **dan** class-nya `error` → berarti itu memang pesan error yang pernah ditambahkan `tampilkanError` sebelumnya, aman untuk dihapus dengan `.remove()`.

Pengecekan `classList.contains("error")` ini penting supaya fungsi ini **tidak salah menghapus** elemen lain yang kebetulan menjadi sibling berikutnya (misalnya `<br>` pada field yang belum pernah error sama sekali).

7.5 Fungsi Utama: `initValidasiForm`

```
function initValidasiForm() {
  const form = document.getElementById("form-tambah");
  if (!form) return;

  form.addEventListener("submit", function (e) {
    let valid = true;
    // ...pengecekan tiap field...
    if (!valid) {
      e.preventDefault();
    }
  });
}
```

- `form` dicari lewat `id="form-tambah"` — ingat dari [bab 2 §2.4](#), `id` ini sengaja dibuat **sama** di kedua form Tambah Buku dan Tambah Anggota.
- `let valid = true;` — beda dengan `const` yang dipakai di fungsi lain ([bab 4 §4.2](#)), `let` dipakai di sini karena nilai `valid` **akan diubah** menjadi `false` kalau ada field yang tidak valid ditemukan saat pengecekan.
- `form.addEventListener("submit", function (e) { ... })` — event `submit` (ingat dari [bab 1 §1.6](#)) terpicu tepat saat tombol `<button type="submit">` ditekan. Perhatikan fungsinya menerima **parameter** `e` (singkatan dari *event*) — objek yang berisi informasi tentang kejadian ini, dan dipakai di [§7.7](#).

7.6 Pola Pengecekan per Field

```
const judul = form.querySelector("[name='judul']", [name='nama']);
if (judul && judul.value.trim() === "") {
  tampilkanError(judul, "Field ini wajib diisi.");
  valid = false;
} else if (nama) {
  hapusError(judul);
}
```

- `form.querySelector("[name='judul']", [name='nama'])` — attribute selector (ingat konsepnya dari [dokumentasi jobsheet-02 §8.5](#)) yang dipisah koma, artinya "cari field yang `name`-nya

judul **atau** nama." Inilah **trik utama** yang membuat satu fungsi `initValidasiForm` bisa menangani **dua form yang berbeda field-nya**: di form Tambah Buku field pertamanya bernama `judul` ([dokumentasi jobsheet-01](#)), di form Tambah Anggota bernama `nama` ([dokumentasi jobsheet-01](#)) — selector ini akan menemukan **salah satu yang benar-benar ada** di form tempat kode ini berjalan, karena hanya satu dari keduanya yang akan cocok tergantung halaman mana yang sedang dibuka.

- `judul.value.trim() === ""` — `.trim()` menghapus spasi kosong di awal/akhir teks (supaya mengetik " " -- hanya spasi -- tetap dianggap kosong, bukan dianggap valid). `=== ""` memeriksa apakah hasilnya benar-benar string kosong.
- Kalau kosong → panggil `tampilkanError` (§7.3) dengan pesannya, lalu set `valid = false` — menandai bahwa form ini **tidak boleh** diproses lebih lanjut.
- Kalau **tidak** kosong (`else if (judul)`) → panggil `hapusError` (§7.4) — membersihkan pesan error lama kalau field ini **sebelumnya** pernah error tapi sekarang sudah diperbaiki pengguna.

Pola yang sama persis diulang untuk field `pengarang`, dengan penyesuaian untuk `tahun` dan `stok`:

```
const tahun = form.querySelector("[name='tahun']");
if (tahun) {
  const nilai = parseInt(tahun.value, 10);
  if (isNaN(nilai) || nilai < 1900 || nilai > 2026) {
    tampilkanError(tahun, "Tahun harus di antara 1900-2026.");
    valid = false;
  } else {
    hapusError(tahun);
  }
}
```

- `parseInt(tahun.value, 10)` — mengubah teks (misalnya "2005") menjadi **angka** sungguhan (2005), dengan 10 menandakan basis bilangan desimal (basis 10 — praktik yang baik untuk selalu disertakan eksplisit, meski untuk kasus umum sering diabaikan).
- `isNaN(nilai)` — NaN adalah singkatan dari *Not a Number*, nilai yang muncul kalau `parseInt` gagal mengubah teks jadi angka (misalnya kalau field dikosongkan). `isNaN()` memeriksa apakah nilai itu memang bukan angka yang valid.
- `nilai < 1900 || nilai > 2026` — mengecek rentang tahun yang masuk akal, **menduplikasi** aturan yang sebenarnya sudah ada di HTML lewat `min="1900" max="2026"` (ingat dari [dokumentasi jobsheet-01 §4.4](#)). Ini **bukan** duplikasi yang sia-sia — dibahas alasannya di §7.8.

Field `stok` memakai pola serupa, hanya memeriksa `nilai < 0` (stok tidak boleh negatif, tanpa batas atas).

7.7 Mencegah Submit Jika Tidak Valid

```
if (!valid) {  
    e.preventDefault();  
}
```

Setelah **semua** field diperiksa, kalau `valid` masih bernilai `false` (artinya minimal satu field bermasalah), baris `e.preventDefault();` dipanggil — method ini **membatalkan** perilaku bawaan event yang sedang terjadi. Untuk event `submit`, perilaku bawaannya adalah "kirim form" (yang di jobsheet ini sebenarnya cuma me-reload halaman, ingat §7.2). Dengan `preventDefault()` dipanggil, halaman **tidak** ikut ter-reload — pengguna tetap berada di form yang sama, dengan pesan-pesan error yang baru saja dimunculkan tetap terlihat, siap untuk diperbaiki.

Kalau `valid` tetap `true` (semua field lolos pengecekan), `e.preventDefault()` **tidak** dipanggil, sehingga form melanjutkan perilaku bawaannya seperti biasa (reload halaman, sesuai kondisi form yang memang belum terhubung ke server sungguhan).

7.8 Kenapa Validasi HTML (`required`, `min`, `max`) Masih Perlu Diduplikasi di JS?

Pertanyaan wajar: kalau HTML sudah punya `required` dan `min/max` sejak jobsheet-01, kenapa perlu ditulis ulang logikanya di JavaScript? Jawabannya sudah disinggung sebagai catatan penting di [README.md](#) jobsheet ini:

Validasi di sini murni client-side dan bisa dilewati (nonaktifkan JS).

Sebenarnya justru **sebaliknya** yang perlu digarisbawahi di sini: validasi bawaan HTML (`required`, `min`, `max`) **maupun** validasi JavaScript **sama-sama** berjalan di **sisi klien** (browser pengguna) — keduanya bisa dilewati oleh pengguna yang cukup paham (misalnya lewat DevTools, atau mengirim request langsung tanpa lewat form). Menduplikasi aturan di JavaScript di jobsheet ini bertujuan untuk **latihan** memakai `insertAdjacentElement`, `classList`, dan manipulasi DOM lain sesuai Sub-CPMK jobsheet ini — **bukan** untuk menambah keamanan sungguhan. Lapisan validasi yang **benar-benar tidak bisa dilewati** adalah validasi **sisi server** (server-side), yang baru akan ditambahkan di Jobsheet 7 — sesuai catatan resmi di [README.md](#) jobsheet ini. Aturan pentingnya: **jangan pernah mengandalkan validasi client-side** saja** untuk data yang benar-benar sensitif/penting.**

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

8. Rangkuman & Latihan Lanjutan

8.1 Rangkuman Keseluruhan Jobsheet 5

Bagian	Fungsi di <code>app.js</code>	Konsep yang Dipelajari
Konsep Dasar JS & DOM	—	<code><script></code> , DOM, <code>getElementById / querySelector</code> , event & event listener
Perubahan HTML	—	Tombol hamburger asli, <code>search-box</code> , <code>class btn-hapus</code> , <code>id="form-tambah"</code>
CSS Pendukung	—	Gaya untuk <code>.error</code> , <code>.search-box</code> , penyesuaian hamburger
Menu Hamburger	<code>initNavToggle</code>	<code>classList.toggle()</code> , event <code>click</code> , guard clause
Konfirmasi Hapus	<code>initHapusConfirm</code>	<code>querySelectorAll + forEach</code> , <code>closest()</code> , <code>confirm()</code> , <code>remove()</code>
Filter Tabel	<code>initTableFilter</code>	Event <code>keyup</code> , <code>textContent</code> , <code>style.display</code> , <code>includes()</code>
Validasi Form	<code>initValidasiForm</code> , <code>tampilkanError</code> , <code>hapusError</code>	Event <code>submit</code> , <code>preventDefault()</code> , <code>createElement</code> , <code>insertAdjacentElement</code> , <code>classList.contains()</code>

8.2 Konsep Inti yang Perlu Diingat

1. **JavaScript menambah lapisan perilaku**, terpisah dari struktur (HTML) dan tampilan (CSS)
 - dihubungkan lewat `<script src="...">` yang diletakkan di akhir `<body>` ([bab 1](#)).
2. **DOM adalah "pohon" objek** yang bisa dibaca dan diubah lewat `getElementById / querySelector / querySelectorAll`, memakai selector CSS yang sama dengan yang sudah kamu kuasai sejak jobsheet-02 ([bab 1 §1.5](#)).
3. **Event listener adalah pola inti interaktivitas**: pilih elemen → `.addEventListener(event, fungsi)` → tulis reaksinya. Tiga event utama di jobsheet ini: `click`, `keyup`, `submit` ([bab 1 §1.6](#)).

4. `classList.toggle() / .contains()` adalah cara modern mengatur status tampilan lewat class CSS, menggantikan trik CSS murni seperti checkbox hack ketika JavaScript sudah tersedia ([bab 4 §4.6](#)).
5. **Guard clause** (`if (!elemen) return;`) penting supaya satu file JavaScript yang sama aman dipakai di banyak halaman berbeda, tanpa error di halaman yang tidak punya elemen tertentu ([bab 1 §1.7](#)).
6. **Validasi client-side bisa dilewati** dan bukan pengganti validasi server-side — ini lapisan kenyamanan pengguna, bukan lapisan keamanan ([bab 7 §7.8](#)).

8.3 Cara Mencoba Sendiri

1. Buka `index.html`, perkecil layar ke mode mobile (DevTools responsif, ingat caranya dari [dokumentasi jobsheet-03 §5.6](#)), lalu klik ikon hamburger — bandingkan rasanya dengan versi checkbox hack di jobsheet-03 (secara visual identik, tapi mekanismenya berbeda total di balik layar).
2. Buka `buku/list.html`, ketik sebagian judul buku (misalnya "bumi") di kolom cari, amati baris lain otomatis hilang. Hapus teksnya lagi, amati semua baris muncul kembali.
3. Klik tombol "Hapus" di salah satu baris — muncul dialog konfirmasi. Klik "Cancel", baris tetap ada. Coba lagi dan klik "OK", baris hilang. Refresh halaman — perhatikan baris itu muncul kembali (ingat catatan di [bab 5 §5.6](#)).
4. Buka `buku/tambah.html`, langsung klik "Simpan" tanpa mengisi apa pun — amati pesan error merah muncul di bawah field yang wajib diisi. Isi salah satu field yang error, klik "Simpan" lagi — amati pesan error field itu hilang, sementara field lain yang masih kosong tetap menampilkan errornya.
5. Buka **DevTools Console** (`F12` → tab *Console*) sambil mencoba langkah 2-4 di atas — perhatikan tidak ada pesan error JavaScript yang muncul, menandakan guard clause dan penanganan elemen sudah benar.

8.4 Ide Latihan Tambahan (Opsional)

1. **Tambah validasi field baru** — misalnya field ISBN di form Tambah Buku (yang saat ini tidak wajib diisi, ingat dari [dokumentasi jobsheet-01 §4.4](#)) validasi supaya hanya menerima angka dan tanda hubung.
2. **Tambah animasi sederhana** pada `initNavToggle` — misalnya tambahkan class CSS `transition` pada `header nav` di `style.css` supaya menu terbuka/tertutup dengan efek geser halus, alih-alih langsung muncul/hilang seketika.
3. **Perluas** `initTableFilter` supaya pencarian bisa dibatasi ke satu kolom saja (misalnya hanya kolom "Judul"), bukan mencari di seluruh teks baris — petunjuk: gunakan

`row.querySelector("td")` seperti pola yang sudah dipakai di [bab 5 §5.4](#), alih-alih `row.textContent`.

4. **Tambah counter jumlah baris tersisa** setelah difilter atau dihapus — tampilkan misalnya "Menampilkan 3 dari 5 buku" di atas tabel, diperbarui setiap kali `initTableFilter` atau `initHapusConfirm` berjalan.

5. **Refactor validasi** — coba ubah `initValidasiForm` supaya nama field yang wajib divalidasi diambil dari sebuah array/daftar, alih-alih menulis blok `if` terpisah untuk tiap field satu-satu (petunjuk: pikirkan pola perulangan `forEach` yang sudah dipakai di [bab 5](#) dan [bab 6](#)).

Kalau ada bagian yang masih membingungkan, coba baca ulang [bab 1](#) sambil mempraktikkan langsung di DevTools Console — ketik `document.querySelector("header nav")` di tab Console salah satu halaman, lalu amati elemen apa yang dikembalikan, supaya konsep "memilih elemen dari DOM" terasa nyata, bukan sekadar teori.